Fairness-aware and Parallelized SFC Placement with VNF Sharing: Implementation, Limitations and Modifications

Advanced Computer Networks – Project Report

23CSB0B03 23CSB0B17 23CSB0B23 23CSB0B56

Abstract

We studied FPPVS, a framework by Zhu, Li and Rezaeipanah [1] that places service function chains (SFCs) in MEC-enabled IoT networks using SFC parallelization, VNF sharing, fairness-aware allocation and federated deep reinforcement learning (FDRL). We built a full Python implementation of it. Our first version accepted only about 10% of requests, worse than a random placement. We traced this to several gaps in the problem as it was specified and in our implementation, and then changed the formulation and the algorithm: Poisson arrivals at a stated offered load, a placement mask that checks compute, bandwidth and latency before a data center is chosen, a per-data-center actor network with a placement prior, honest accounting for shared VNF instances, rollback-and-retry admission, and corrected federated training. On synthetic Backbone (24 nodes) and Metro (17 nodes) topologies at three heavy load levels (offered demand at 90%, 100% and 110% of network capacity), the updated FPPVS accepts between 76.6% and 91.0% of requests on a hold-out stream (mean of 5 seeds). Under the same conditions, the original FPPVS as specified in the paper accepts 40.9–50.2%, a 2022 SFC-placement method by Zhang et al. accepts 43.0–53.1%, and greedy and random placement stay between 49% and 59%. Updated FPPVS holds about half the link bandwidth per running service of greedy and random, 19% less than the original FPPVS, and about the same as the 2022 adaptation.

How to read this report. Material that describes the original paper is written as "the paper" or "the original work". Everything our team designed, changed or measured is marked as ours and is summarised in Table 12.

1 Introduction

1.1 Problem addressed

Network Function Virtualization (NFV) replaces dedicated middleboxes, such as firewalls (FW), intrusion detection (IDS), deep packet inspection (DPI), load balancers (LB) and NAT, with software Virtual Network Functions (VNFs). A network service is an ordered list of VNFs called a Service Function Chain (SFC). In a Mobile Edge Computing (MEC) network the VNFs run on small edge data centers (DCs) that have limited compute, and the DCs are connected by links with limited bandwidth and non-zero latency. The *SFC placement problem* asks, for every arriving request, on which DC each VNF should run and how traffic should be routed between them, so that capacity, bandwidth and the request's latency limit are respected. The problem is NP-hard and has to be solved online, because requests arrive one after another and leave after their lifetime.

1.2 Motivation

Two effects make the problem harder in IoT/MEC settings. First, end-to-end latency grows with chain length when every packet passes through every VNF in sequence. Second, many requests compete for the same small edge DCs, so a placement policy that only looks at the current request can starve later ones. Existing methods tend to address only one of these at a time: network function

parallelism reduces latency, while VNF reuse reduces resource use. Fairness between competing flows is usually ignored.

1.3 Main contributions of the original paper

The paper [1] proposes FPPVS, which combines four ideas:

1. **SFC parallelization (Algorithm 1).** VNFs are split into *shapers*, which modify packets and must stay in order, and *monitors*, which only read packets. Monitors are moved onto branches that run in parallel with the main chain, and a dependency matrix says where each branch must rejoin. The sequential SFC (s-SFC) becomes a directed acyclic graph (p-SFC), and latency is measured on its critical path.
2. **VNF sharing with demand prediction.** Before creating a new VNF instance, the method tries to reuse a compatible live instance. A bounded SFC queue network (SQN) and an LSTM forecast upcoming demand per VNF type.
3. **Fairness-aware allocation.** A concave α-fair utility over flow throughput is part of the objective, so a few heavy flows cannot take all the resources. Jain's fairness index (JFI) is reported.
4. **Federated actor-critic learning.** Local agents with a CNN actor-critic place VNFs, and a cloud server combines their models with federated averaging (FedAvg) [2]. Exploration uses a reward-driven adaptive ϵ-greedy rule.

The paper reports 88.1% service acceptance on a backbone topology and 75.9% on a metropolitan topology, ahead of four baselines (RVNF, DRL-SFCP, PVFP and DPC).

2 Limitations of the Original Work

We list the limitations we found while implementing the paper. Some are gaps in what the paper specifies, so it cannot be reproduced exactly. Others are design problems that showed up once the method ran. Table 1 states which ones our project addresses.

L1. The workload is under-specified. The paper fixes a mean service lifetime of 1000 time units and 4000 requests, but it does not give the arrival rate. The arrival rate decides how many SFCs are active at the same time, and so it decides the acceptance ratio. When we used one arrival per time unit, about 1000 SFCs would have to be active at once, while a 24-node network with 10–20 compute units per DC can only hold about 70–100.

L2. Datasets and code are not available. The USAnet and Italian operator topologies, the traffic traces, the MATLAB code and the baseline configurations are not published. Results can only be compared in spirit.

L3. Feasibility is checked too late. The paper prunes nodes that lack compute, but bandwidth and the latency limit are checked only after the whole chain is placed. An agent can therefore spend a full episode building a placement that is rejected at the last step.

L4. The sharing model can overbook capacity. The paper does not say how much capacity a shared instance reserves or when it is released. A natural reading, which our first version followed (instances sized at three times the demand, only one times reserved), lets the pool use capacity that the DC does not have.

L5. The CNN cannot tell data centers apart. The paper's actor is a CNN over a feature grid followed by fully connected layers, with one output per DC. After global pooling, the network no longer knows which column belongs to which DC, so it can only learn fixed DC preferences.

L6. Each episode is one request. The reward only sees the current request. Effects on later requests, such as fragmentation or a saturated link, reach the agent only indirectly.

L7. Resource efficiency is reported only as utilization. Higher utilization can mean better packing, but it can also mean more waste per request. The paper does not report resources held per served SFC over time.

Table 1: Limitations and how our project treats them.

	Our treatment	Status
L1	Poisson arrivals whose rate is set from an explicit offered load ρ	Addressed
L2	Synthetic graphs of the same size; simple, documented baselines	Partly (data still missing)
L3	Look-ahead mask on compute, every routed link and the latency bound	Addressed
L4	Per-instance overhead model, all usage counted against physical capacity	Addressed
L5	Per-column actor with shared weights plus a learnable placement prior	Addressed
L6	Placement prior with future-hop term; rollback-and-retry admission	Partly
L7	New metrics: resources held per running SFC and resource-time per accepted SFC	Addressed

Problems in our first implementation. Before the modifications, our implementation of the paper had its own bugs. The package did not import (a queue-memory class had been deleted), training crashed when every action was masked, federated agents replayed the same ten requests in every round and shared one resource ledger, and the state ignored resources held by earlier requests. With these bugs and limitations L1–L5, it accepted 10.4% of requests, less than greedy (15.8%) and random (11.6%) placement. A diagnosis run showed that every rejection was due to link bandwidth running out.

3 System Model and Architecture

3.1 Substrate network

The MEC network is an undirected graph $G_s = (V_s, E_s)$. Each DC $v_i \in V_s$ has compute capacity C_i. Each link $(v_i, v_j) \in E_s$ has latency L_{ij} and bandwidth capacity B_{ij}. Traffic between two DCs follows the latency-shortest path $P(v_i, v_j)$. Two VNFs on the same DC exchange traffic at no bandwidth or network latency cost. We use two topologies with the sizes used in the paper: *Backbone* (24 DCs, 47 links, like USAnet) and *Metro* (17 DCs, 29 links, like the Italian access network).

3.2 Requests supported

A request u is the tuple $(s_u, \mathcal{F}_u, d_u, D_u, t_u, \tau_u, r_u, L_u^{\max})$: source DC, ordered VNF list, destination DC, dependency matrix, arrival time, lifetime (TTL), data rate and maximum end-to-end latency. Each VNF $f \in \mathcal{F}_u$ has a type, a kind (shaper or monitor), a compute demand c_f and a processing latency ℓ_f. The implementation supports:

- **Five VNF types**: IDS, DPI, LB, FW and NAT. IDS and DPI can act as monitors; the others are shapers.
- **Purely sequential chains** (only shapers), **purely parallel chains** (only monitors, all branching from the source), and **mixed chains** in which a monitor must finish before a later shaper.
- **Chains of 4 to 10 VNFs** with heterogeneous compute demand and a heavy-tailed (power-law) data rate.

For each request the system outputs: an accept/reject decision; if accepted, the DC for every VNF, the substrate route of every virtual link, the latency split into network, virtualization and

processing parts on the critical path, the compute and bandwidth reserved, and which VNF instances were reused or started. If rejected, it outputs the reason (for example, no DC fits the next VNF).

3.3 Architecture

Figure 1 shows the components. Blue blocks follow the paper directly, orange blocks are paper ideas that we re-implemented, and green blocks are new or changed by our team.

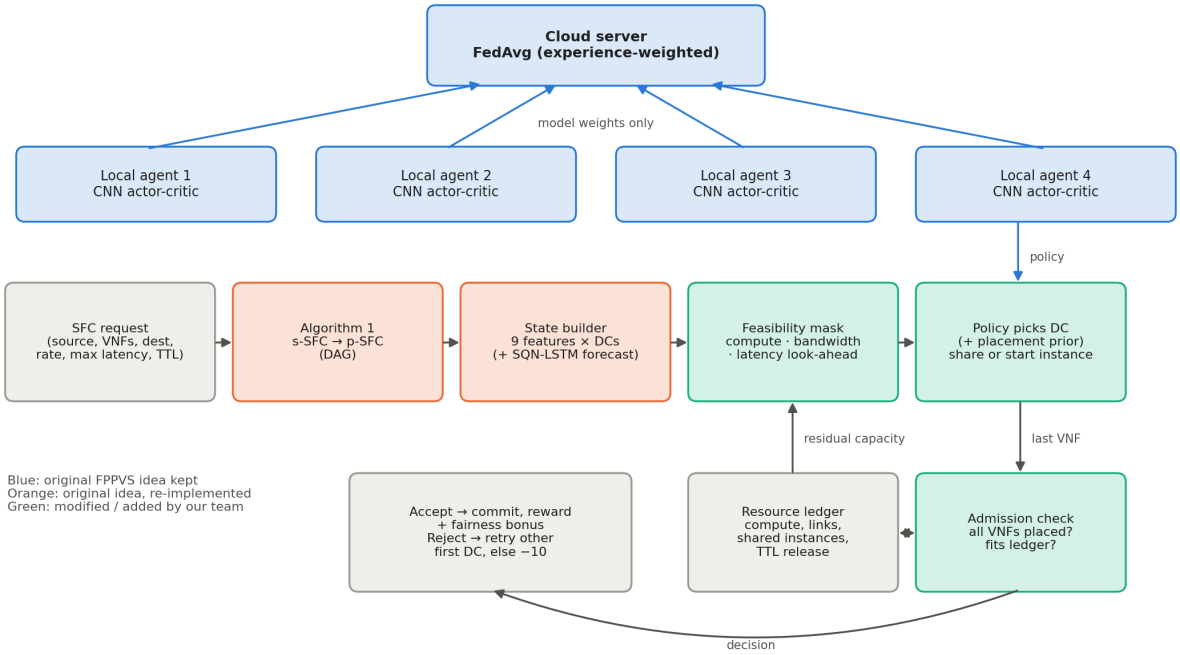


Figure 1: System architecture. A request is converted to a p-SFC, its VNFs are placed one at a time by a local agent under a feasibility mask, and the admission check commits or rejects the whole chain against the resource ledger. Agents share model weights through FedAvg, never raw request data.

- **Algorithm 1 (s-SFC to p-SFC).** Figure 2 shows the output of our implementation for a five-VNF chain. FW, NAT and LB stay on the main chain. IDS branches off after FW and must rejoin before NAT because the dependency matrix says so. DPI has no dependent shaper, so it rejoins at the destination.
- **State builder and SQN-LSTM.** Builds a grid with one column per DC and nine feature rows (Section 5). The SQN keeps the per-type demand of the last five requests, and the LSTM predicts the next one.
- **Feasibility mask, policy and admission** are described in Section 5.
- **Resource ledger.** Holds the compute, link bandwidth and live shared instances of all active SFCs, releases them when a TTL expires, and integrates resource use over time for the metrics.
- **Federated learning.** Four local agents, each with its own request stream and ledger, train locally; the cloud averages their weights.

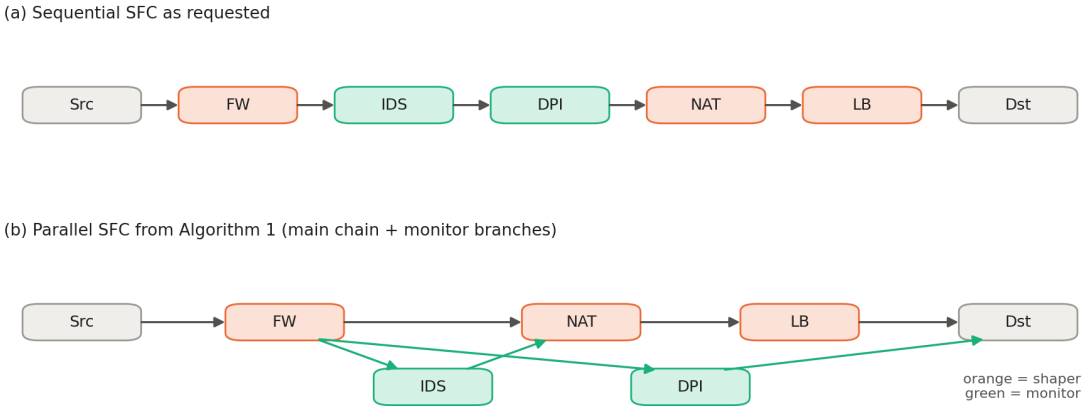


Figure 2: Algorithm 1 as implemented. Top: requested sequential chain. Bottom: generated p-SFC. IDS must finish before NAT; DPI has no dependent shaper and rejoins at the destination.

4 Modified Problem Formulation

4.1 Original formulation (paper)

The paper minimizes a weighted sum over time,

$$F_{obj} = \min \sum_t \xi_{cost} O_{cost}(t) + \xi_{E2E} O_{E2E}(t) - \xi_{acc} O_{acc}(t) - \xi_{utility} O_{utility}(t), \tag{1}$$

where $O_{acc} = \frac{1}{|U|} \sum_u y_u$ is the acceptance ratio ($y_u = 1$ if request u is accepted), O_{cost} is deployment cost, O_{E2E} is end-to-end latency and $O_{utility}$ is the α-fair throughput utility. Constraints cover DC capacity, link bandwidth, latency, placement uniqueness and VNF dependencies.

4.2 Our modifications

M1. Explicit offered load (L1). Requests arrive as a Poisson process with rate λ, and lifetimes are exponential with mean $\bar{\tau} = 1000$. We set λ from a target offered load

$$\rho = \frac{\lambda \, \bar{\tau} \, \bar{c}}{\sum_i C_i}, \qquad \bar{c} = 7 \times 0.7 = 4.9, \tag{2}$$

where $\bar{c}$ is the mean compute demand of one SFC (4–10 VNFs of 0.5–0.9 units each). $\rho = 1$ means the average offered compute equals the total compute of the network. Our main scenarios use heavy load, $\rho \in \{0.9, 1.0, 1.1\}$. At light load the problem is easy: at $\rho = 0.5$ almost any placement that keeps chains compact accepts close to every request, so light load says little about the method. We still report $\rho = 0.5$, and the overload points $\rho \in \{1.25, 1.5\}$, in a load sweep, so the full range is visible.

M2. Instance-based compute model (L4). Each live VNF instance costs a fixed overhead o (its VM or container footprint, $o = 0.25$) plus the demand of every flow it serves. Let $\mathcal{A}(t)$ be the set of active requests and $\mathcal{I}_i(t)$ the set of live instances on DC v_i. The compute constraint is

$$\sum_{u \in \mathcal{A}(t)} \sum_{f \in \mathcal{F}_u} c_f \, x^u_{f,i} \; + \; o \, |\mathcal{I}_i(t)| \; \leq \; C_i \qquad \forall v_i, t, \tag{3}$$

where $x^u_{f,i} = 1$ if VNF f of request u runs on v_i. With sharing, an instance of type k on v_i is created by the first VNF of that type and removed when its last user leaves. Without sharing, every VNF has its own instance.

M3. Bandwidth and latency on the p-SFC. For every virtual edge (a, b) of the p-SFC, the data rate r_u is reserved on each link of $P(\pi(a), \pi(b))$, where π is the placement:

$$\sum_{u \in \mathcal{A}(t)} \sum_{(a,b) \in E_u} r_u \mathbb{1}\left[(v_i, v_j) \in P(\pi(a), \pi(b))\right] \leq B_{ij}. \tag{4}$$

The latency of a request is the longest source-to-destination path in the p-SFC, counting link latency and VNF processing: $L_u = \max_p \sum_{(a,b) \in p} \left(L_{P(\pi(a),\pi(b))} + \ell_b\right) \leq L_u^{\max}$.

M4. Objective. The main objective stays the same as the paper: maximize O_{acc}. As a second objective we minimize the resources held per served request over time (L7):

$$R_{comp} = \frac{\int_t \sum_i \text{used}_i(t)\, dt}{\#\text{accepted}}, \qquad R_{bw} = \frac{\int_t \sum_{(i,j)} \text{used}_{ij}(t)\, dt}{\#\text{accepted}}. \tag{5}$$

Latency and fairness enter through the reward (Section 5.5), as in the paper. We do not model monetary cost separately, because with a fixed price per unit it is proportional to R_{comp}.

5 Proposed Approach

5.1 Overview

Each request is one episode. Its VNFs are placed one at a time in their original order, which is also a topological order of the p-SFC. When VNF f is placed on a candidate DC, every virtual edge that enters f (and its edge to the destination, if it has one) can already be routed. This is what makes an early feasibility check possible. Figure 3 and Algorithm 1 show one decision.

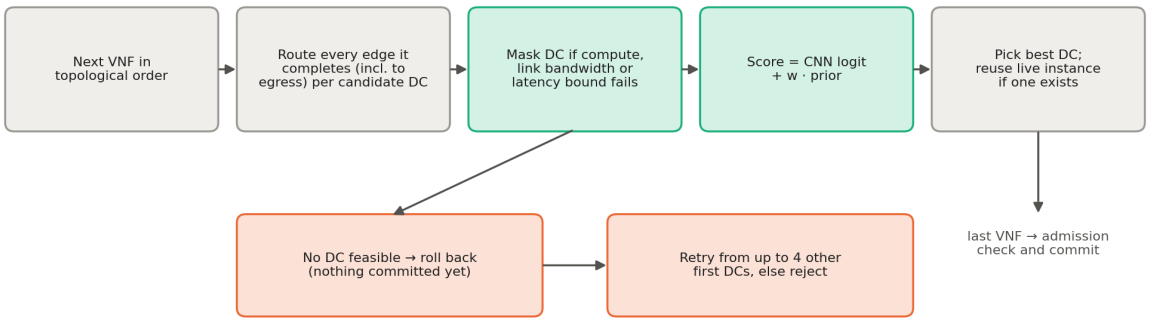


Figure 3: One placement decision in the updated FPPVS.

Algorithm 1 Updated FPPVS admission for one request u

Require: request u, ledger $\mathcal{L}$, policy π_θ, retries K

$G_u \leftarrow$ PARALLELIZE(u) ▷ Algorithm 1 of the paper
for attempt $= 0, \ldots, K$ **do**
  reset episode state (nothing is committed yet)
  for each VNF f of u in order **do**
    for each DC v **do**
      route the edges of G_u completed by putting f on v
      $m_v \leftarrow$ compute, bandwidth and latency bound all hold
    end for
    if no m_v is true **then**
      break ▷ dead end, try next attempt
    end if
    if f is the first VNF and attempt > 0 **then**
      $v^* \leftarrow$ the attempt-th best DC by prior score, excluding the policy's choice
    else
      $v^* \leftarrow \arg\max_{v:\, m_v} \left(\text{logit}_\theta(s)_v + w \cdot \text{prior}_v\right)$
    end if
    place f on v^*; reuse a live instance of its type on v^* if sharing is on
  end for
  if all VNFs placed **and** EVALUATE(G_u) feasible **and** fits $\mathcal{L}$ **then**
    commit to $\mathcal{L}$ until $t_u + \tau_u$; **return** accept
  end if
end for
return reject

5.2 MDP: state, action and mask

State. A $9 \times |V_s|$ grid, one column per DC (Table 2).

Table 2: State features per DC column. Rows marked "ours" were added by us.

Row	Feature	Origin
1	Residual compute fraction (includes all active SFCs)	paper, fixed by us
2	Demand of the current VNF / capacity	paper
3	Latency from the DC of the previous VNF	paper
4	Latency to the destination	ours
5	Smallest remaining bandwidth fraction on newly routed links	ours
6	A live instance of this VNF type exists (reuse possible)	ours
7	SQN-LSTM demand forecast for this VNF type	paper
8	Placement prior score	ours
9	Feasibility mask	ours

Action. Choose one DC for the current VNF. **Mask.** A DC is allowed only if (i) its residual compute covers the VNF demand plus instance overhead when a new instance is needed, (ii) every link on the newly routed paths still has the data rate free, and (iii) the critical-path latency so far plus the shortest latency from the DC to the destination is within $L_u^{\max}$. Because shortest-path latency obeys the triangle inequality, (iii) never removes a DC that could still lead to a valid placement.

5.3 Placement prior

The prior score of DC v for VNF f is

$$\begin{aligned}\text{prior}_v = & -(h_v^{new} + h_v^{dst}) - 2\,\frac{T_v + L(v, d_u)}{L_u^{\max}} + 0.5\,\mathbf{1}[\text{reuse}] \\ & + 0.3\,\mathbf{1}[v = v_{prev}] + 0.5\,\tilde{C}_v - 3\,\frac{\max(0,\, 0.25 - \beta_v)}{0.25},\end{aligned} \tag{6}$$

where h_v^{new} is the number of link hops (weighted by data rate) the new edges use, h_v^{dst} the hops from v to the destination, T_v the critical-path finish time of f on v, $\tilde{C}_v$ the residual compute fraction after placement and β_v the bandwidth margin. In words: keep the chain short and on few DCs, reuse live instances, avoid nearly full DCs and links.

5.4 Actor-critic network

The paper's three convolutional layers with 64 filters, batch normalization and ReLU are kept, but the kernels are 3×1, running along the feature axis only, because neighbouring columns are unrelated DCs. The **actor** applies the same two 1×1 convolutions to every column, averages over the feature axis and outputs one logit per DC; we add $w \cdot \text{prior}$, where w is a learnable scalar (initial value 2). The network therefore learns a correction on top of a sensible rule, and it is permutation-equivariant: reordering the DCs reorders the outputs in the same way (this is checked by a unit test). The **critic** keeps the paper's design: global pooling, a 256-unit fully connected layer, dropout 0.3 and one output.

5.5 Reward

Following Eq. (28) of the paper: $+1$ for each valid placement, -1 when a new instance must be started, and -10 if the request is rejected. We added a small bandwidth term (-0.2 per data-rate-weighted hop) and dropped the paper's $+5$ reuse bonus, because the prior already rewards reuse. When the request is accepted, the agent gets $5\,(1 + \max(0, 1 - L_u/L_u^{\max}))$, which rewards latency slack, plus a fairness bonus $5\,\Delta\text{JFI}$ (the change in Jain's index [5] over the throughputs of accepted requests).

5.6 Training

Each episode is trained with advantage actor-critic [4]: discounted returns, advantage = return − critic value, actor loss with an entropy bonus, critic MSE loss, and gradient clipping. Exploration combines the paper's adaptive ϵ-greedy rule (ϵ shrinks after an accepted request and grows after a rejection) with small Gaussian noise on the logits. Four agents train locally for 50 episodes per round on their own request streams and ledgers, continuing through the stream from round to round; the server then averages their weights, weighted by the number of episodes. Evaluation is deterministic (arg-max over allowed DCs) on a hold-out stream that is never used for training. Table 3 lists the hyperparameters.

Table 3: Training hyperparameters.

Optimizer	Adam, lr 5×10^{-4}	Discount γ	0.99
Federated agents	4	FedAvg rounds	10
Episodes per agent per round	50	Total training episodes	2000
ϵ start / min / decay	0.15 / 0.02 / 0.995	Logit noise std	0.10
Entropy coefficient	0.01	Gradient clip	1.0
Critic loss weight	0.5	Prior weight w (initial)	2.0 (learned)
SQN depth / LSTM hidden	5 / 64	Retries K	4
Instance overhead o	0.25 units	Training requests per agent	600

6 Performance Evaluation

6.1 Experimental setup

Hardware: laptop with a 12th Gen Intel Core i5-12500H CPU and 16 GB of RAM, running Windows 11. No GPU was used. **Software:** Python 3.12.1, PyTorch 2.10 (CPU build), NetworkX 3.6.1 and Matplotlib 3.10.8. Scenarios run in parallel worker processes with one PyTorch thread each. Each scenario is repeated with 5 seeds (7–11). For each seed we generate a new network, four training streams and one hold-out stream. Training one FPPVS model takes about 60 s.

Table 4: Simulation parameters. Values follow Section 5.1 of the paper unless marked as ours.

Parameter	Value
Topologies	Backbone: 24 DCs, 47 links; Metro: 17 DCs, 29 links (random connected graphs)
DC compute capacity	$\mathcal{U}(10, 20)$ units
Link bandwidth	100–200 units in steps of 20
Link latency	$\mathcal{U}(5, 20)$ ms
SFC length	4–10 VNFs, types IDS, DPI, LB, FW, NAT
Monitor probability (IDS/DPI)	0.45; a monitor gates a later shaper with prob. 0.5
VNF compute demand	$\mathcal{U}(0.5, 0.9)$ units (500–900 Mbps normalized)
VNF processing latency	$\mathcal{U}(0.01, 0.2)$ ms
Data rate per request	bounded power law on $[5, 30]$, exponent 1.5
Maximum latency	$\mathcal{U}(100, 150)$ ms
Lifetime (TTL)	exponential, mean 1000 time units
Arrivals	Poisson, rate set by offered load ρ
Offered load ρ	0.9, 1.0, 1.1 (main); 0.5, 1.25, 1.5 (load sweep)
Instance overhead	0.25 units per live instance
Requests per stream	600 hold-out; 600 per training agent

6.2 Performance metrics

- **Service acceptance**: accepted hold-out requests divided by all hold-out requests (higher is better).
- **Compute / bandwidth per running SFC**: time-averaged compute (or bandwidth) in use divided by the time-averaged number of running SFCs. This is the amount of resources one service holds at any moment (lower is better).
- **Compute·time and bandwidth·time per accepted SFC**: R_{comp} and R_{bw} from Section 4 (lower is better).
- **Instances per accepted SFC**: VNF instances started divided by accepted requests; this measures how much sharing is used (lower is better).
- **Latency**: mean critical-path end-to-end latency of accepted requests.
- **Decision time**: wall-clock time per request, including retries.

6.3 Compared methods

We compare five methods. All of them run in the same simulator, on the same networks, training streams and hold-out requests, with the same instance model (M2), so none of them can overbook capacity. The three learning methods get the same training budget of 2000 episodes.

- **Updated FPPVS (ours):** the method of Section 5.
- **Original FPPVS** (Zhu et al. [1]), run as the paper describes it: the paper's four-row state (residual capacity, VNF demand, distance from the previous VNF, SQN-LSTM forecast),

pruning by compute only, the paper's CNN (three 3×3 convolution layers with 64 filters, global pooling, a 256-unit layer with dropout 0.3, one output per DC), the paper's step reward (Eq. 28: $+1$ valid, $+5$ reuse, -1 new instance, -10 rejection), adaptive ϵ-greedy exploration and federated training (4 agents, 10 rounds, 50 episodes). No look-ahead mask, no prior and no retries.

- **2022 adaptation** of Zhang et al. [3]: online SFC placement with sequential VNF placement (no parallelization), reuse of an existing same-type VNF, an SQN-LSTM queue of depth $T_D = 20$ for demand forecasting, an actor-critic placement policy (a fully connected network over the same four-row state), and rollback when a request cannot be completed. Its reward follows the long-term revenue-to-cost ratio (LTCR): on success the agent receives $10 \cdot \min(2, \text{revenue/cost})$, where revenue is the requested compute plus bandwidth and cost is the compute and link bandwidth actually used; a failure gets a fixed penalty of -10. It is trained by a single synchronous agent. This is our adaptation from the method description; the authors' code was not available.
- **Greedy:** the feasible DC with the most residual compute (load balancing), without sharing.
- **Random:** a random feasible DC, without sharing.

Greedy and random use our feasibility mask, so they never pick a DC that is already known to be infeasible. We also compare against the numbers published in the FPPVS paper (Section 6.4). Separately, an ablation study switches off one part of updated FPPVS at a time (no sharing, no fairness bonus, a single agent, no retries, and the placement prior alone without learning).

6.4 Results

A. Service acceptance

Table 5 and Figure 4 give hold-out acceptance for every scenario. The main scenarios are heavy load: offered demand at 90%, 100% and 110% of the network's compute capacity. Updated FPPVS reaches the 70% target in all six main scenarios. Its mean acceptance ranges from 76.6% (Metro 1.1) to 91.0%, with an average of 83.7%. The lowest value in any single seed was 73.5%, so the target also holds run by run, not only on average.

Table 5: Hold-out service acceptance (%), mean ± standard deviation over 5 seeds. Best value per row in bold.

Scenario (topology, load)	Updated (ours)	Original FPPVS	2022 adaptation	Greedy	Random
Backbone 0.9	**91.0 ± 3.6**	45.0 ± 4.8	47.8 ± 7.6	58.7 ± 4.4	57.4 ± 4.0
Backbone 1	**86.4 ± 3.7**	43.6 ± 5.1	44.5 ± 6.5	53.4 ± 5.1	55.3 ± 3.4
Backbone 1.1	**80.8 ± 3.8**	40.9 ± 5.3	43.0 ± 10.4	51.9 ± 4.9	52.9 ± 3.3
Metro 0.9	**86.1 ± 2.9**	50.2 ± 5.8	53.1 ± 9.3	56.3 ± 3.1	56.6 ± 3.0
Metro 1	**81.3 ± 3.5**	49.7 ± 4.5	49.4 ± 9.7	51.8 ± 3.6	52.6 ± 3.0
Metro 1.1	**76.6 ± 2.5**	47.2 ± 5.0	50.1 ± 5.2	48.8 ± 1.9	50.9 ± 3.1
Other load levels (light load and overload)					
Backbone 0.5	**99.6 ± 0.4**	58.1 ± 4.1	68.1 ± 5.4	72.1 ± 5.8	73.2 ± 4.5
Backbone 1.25	**75.2 ± 2.9**	38.7 ± 3.9	41.1 ± 3.0	49.3 ± 3.8	50.9 ± 2.7
Backbone 1.5	**65.2 ± 3.3**	35.0 ± 4.2	38.4 ± 4.3	43.4 ± 4.1	44.2 ± 2.1
Metro 0.5	**99.0 ± 0.8**	64.6 ± 10.2	74.0 ± 9.7	71.4 ± 3.0	72.1 ± 2.9
Metro 1.25	**69.2 ± 4.2**	44.0 ± 2.9	40.5 ± 7.8	47.2 ± 2.1	46.9 ± 2.3
Metro 1.5	**60.6 ± 1.9**	36.7 ± 5.0	42.2 ± 3.9	42.2 ± 2.7	43.1 ± 1.3

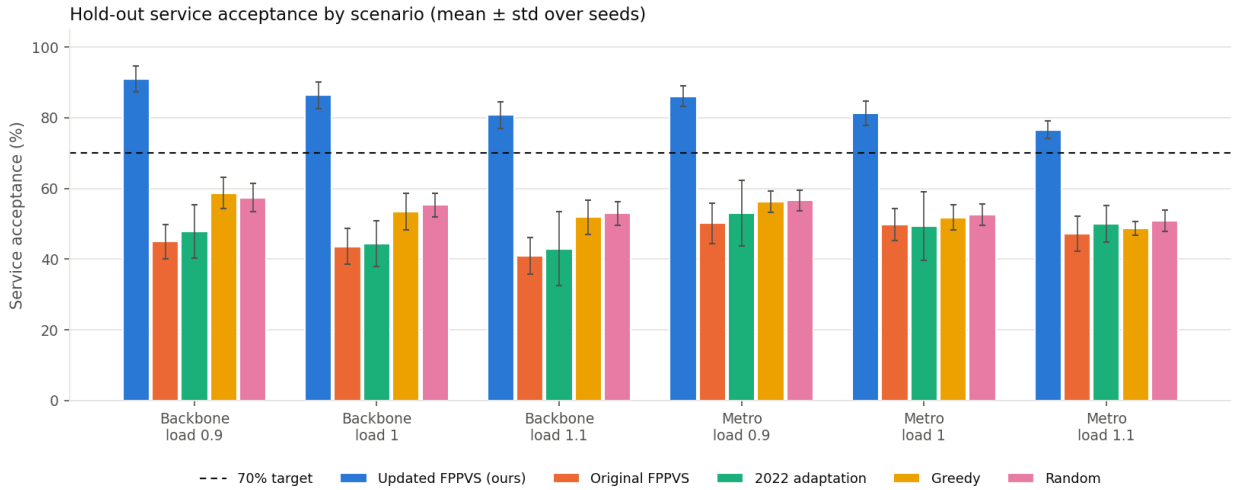


Figure 4: Hold-out acceptance in the six main scenarios. Error bars show one standard deviation over seeds; the dashed line is the 70% target.

Observations.

- Updated FPPVS is the best method in every scenario. At full load ($\rho = 1$) it accepts 86.4% on Backbone and 81.3% on Metro, and at 10% overload ($\rho = 1.1$) still 80.8% and 76.6%.
- The original FPPVS reaches 40.9–50.2% in the main scenarios (mean 46.1%), and the 2022 adaptation reaches 43.0–53.1% (mean 48.0%). Updated FPPVS is 37.6 points above the original and 35.7 points above the 2022 adaptation on average.
- Both learning baselines score below greedy and random in most scenarios. The reason is how infeasible choices are handled. Greedy and random run with our feasibility mask, so they never pick a DC that is already known to break bandwidth or latency. The original FPPVS and the 2022 adaptation check only compute while placing, as their methods describe, and find bandwidth and latency violations only when the whole chain is done (see Part E).
- The 2022 adaptation is the least stable method: its standard deviation across seeds reaches 10 points (Backbone, $\rho = 1.1$). One reward value per episode (LTCR) gives the agent a weak learning signal.
- Greedy and random accept 49–59%. They spread VNFs over many DCs, so each request uses about twice as much link bandwidth as updated FPPVS (Section C), and links run out first.
- Metro is harder than Backbone at the same load. It has fewer DCs and fewer links, so there are fewer alternative paths when a link fills up.

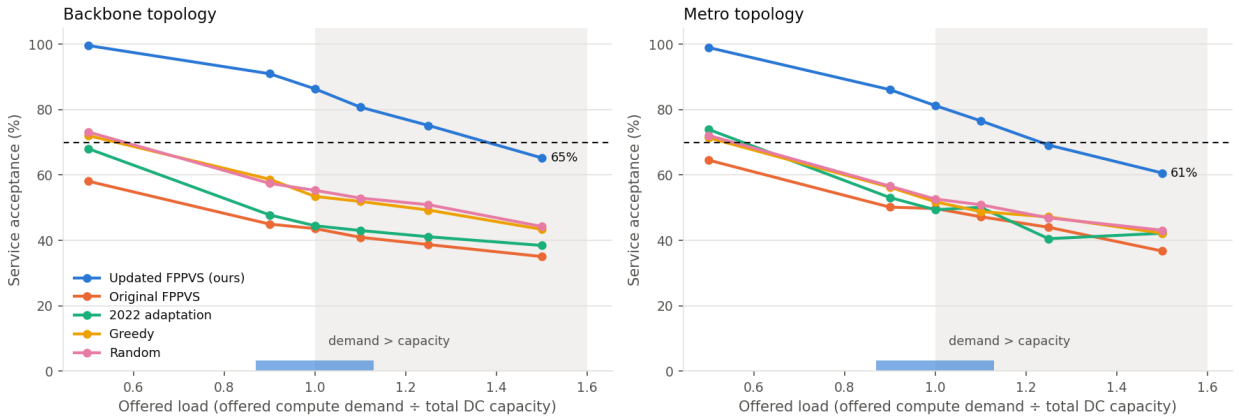


Figure 5: Acceptance against offered load, from light load (0.5) to 50% overload (1.5). The shaded area is where the average offered compute is larger than the total compute of the network; the blue mark on the x-axis shows the main scenarios.

Load sweep. Figure 5 shows the full range of loads. At light load ($\rho = 0.5$) updated FPPVS accepts 99.6% (Backbone) and 99.0% (Metro). This is expected: with the network only half full, almost every request fits as long as chains are kept compact. That is why we use heavier loads as the main scenarios. As load rises, acceptance falls smoothly: 75.2% and 69.2% at $\rho = 1.25$, and 65.2% and 60.6% at $\rho = 1.5$. All other methods stay at or below 51% in this overload region. Below 70% is unavoidable there: at $\rho = 1.5$ only about two thirds of the offered compute can fit in the network at any time, whatever the placement policy.

B. Ablation study

Table 6: Ablation: hold-out acceptance (%) of updated FPPVS with one part switched off at a time.

Scenario	Full	w/o sharing	w/o fairness	1 agent	w/o retry	Prior only
Backbone 0.9	91.0 ± 3.6	78.7 ± 3.7	91.4 ± 3.2	90.9 ± 2.0	90.9 ± 2.8	91.9 ± 3.1
Backbone 1	86.4 ± 3.7	73.8 ± 3.1	85.6 ± 3.9	86.0 ± 3.4	85.4 ± 3.1	86.3 ± 2.8
Backbone 1.1	80.8 ± 3.8	68.2 ± 2.7	80.4 ± 4.1	81.2 ± 3.2	80.2 ± 3.5	80.9 ± 3.0
Metro 0.9	86.1 ± 2.9	76.1 ± 2.1	86.8 ± 3.1	86.2 ± 3.0	86.0 ± 2.9	86.1 ± 3.1
Metro 1	81.3 ± 3.5	70.6 ± 3.2	81.1 ± 2.9	81.0 ± 3.2	81.9 ± 4.0	81.7 ± 3.2
Metro 1.1	76.6 ± 2.5	64.5 ± 2.8	75.7 ± 3.8	75.9 ± 3.8	76.4 ± 3.7	76.4 ± 2.9

Table 6 switches off one part of updated FPPVS at a time.

- **VNF sharing matters most.** Without it, acceptance falls by 10–13 points in every main scenario (for example 86.4% to 73.8% on Backbone at $\rho = 1$). Every VNF then pays its own instance overhead, so the DCs fill up sooner.
- **Fairness bonus, single vs. federated training and retries** change acceptance by about one point or less, which is within the seed-to-seed spread. Fairness is meant to balance throughput between flows, not to raise acceptance, so this is expected. Retries help only in the rare case where the first placement hits a dead end.
- **The learned network adds little over its prior.** "Prior only" uses the same mask, prior and retries with no neural network, and it is within about one point of the full method everywhere. We report this openly: in our experiments the gains come mainly from the problem modifications (mask, sharing model, prior), not from the reinforcement learning itself. One reason is limitation L6: each episode covers only one request, so the agent sees little of the long-term effect of its choices.

C. Resource usage

Figure 6 and Table 8 show the resources each method holds for the services it runs. Table 7 summarises the difference between updated FPPVS and each other method, averaged over the six main scenarios.

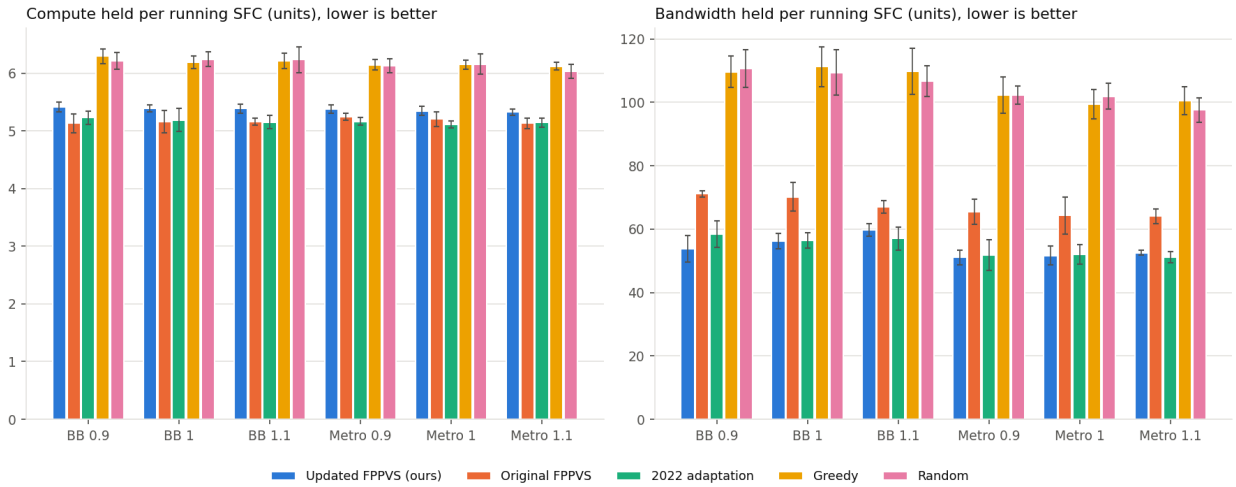


Figure 6: Compute (left) and bandwidth (right) held per running SFC, averaged over time. Lower means each service costs less to keep running.

Table 7: Average reduction in resource use achieved by updated FPPVS over each method (main scenarios; positive means updated FPPVS uses less, negative means it uses more).

Reduction vs.	Compute per SFC	Bandwidth per SFC	BW·time per accepted	Instances per accepted
Original FPPVS	-3.9%	+19.2%	+18.9%	+13.6%
2022 adaptation	-4.0%	+0.5%	+2.1%	+13.7%
Greedy	+13.1%	+48.7%	+49.4%	+91.8%
Random	+12.9%	+48.3%	+48.6%	+91.8%

Table 8: Resource use and latency per scenario (mean over seeds; best per column within a scenario in bold). "Per acc." means per accepted SFC.

Scenario	Method	Compute per SFC	Bandwidth per SFC	Compute·time per acc. (10^3)	BW·time per acc. (10^3)	Instances per acc.	Latency (ms)
Backbone 0.9	Updated (ours)	5.42	**53.8**	5.43	**54.0**	**0.57**	**46.5**
	Original FPPVS	**5.14**	71.2	**5.08**	70.5	0.59	65.6
	2022 adaptation	5.23	58.4	5.37	60.0	0.59	66.2
	Greedy	6.29	109.6	6.31	109.8	6.66	107.8
	Random	6.22	110.7	6.40	114.0	6.57	108.8
Backbone 1	Updated (ours)	5.39	**56.2**	5.42	**56.5**	**0.53**	**50.0**
	Original FPPVS	**5.16**	70.2	**5.20**	70.8	0.59	64.2
	2022 adaptation	5.19	56.5	5.28	57.4	0.61	67.2
	Greedy	6.19	111.2	6.40	115.1	6.58	107.7
	Random	6.24	109.4	6.27	110.1	6.58	108.1
Backbone 1.1	Updated (ours)	5.39	59.6	5.40	59.9	**0.52**	**53.2**
	Original FPPVS	5.16	66.9	**5.17**	67.1	0.63	63.6
	2022 adaptation	**5.15**	**57.0**	5.26	**58.2**	0.63	70.2
	Greedy	6.21	109.9	6.30	111.5	6.59	107.4
	Random	6.23	106.7	6.24	107.0	6.56	108.8
Metro 0.9	Updated (ours)	5.38	**51.1**	5.50	**52.3**	**0.55**	**45.8**
	Original FPPVS	5.24	65.5	5.42	67.8	0.64	62.0
	2022 adaptation	**5.17**	51.8	**5.28**	53.1	0.67	65.5
	Greedy	6.15	102.2	6.27	104.6	6.56	105.2
	Random	6.13	102.2	6.22	103.9	6.47	105.9
Metro 1	Updated (ours)	5.35	**51.6**	5.44	**52.5**	**0.55**	**48.6**
	Original FPPVS	5.21	64.3	**5.28**	65.4	0.67	62.2
	2022 adaptation	**5.11**	52.0	5.35	54.4	0.62	67.2
	Greedy	6.15	99.4	6.46	104.4	6.53	104.1
	Random	6.16	101.9	6.38	105.6	6.51	105.2
Metro 1.1	Updated (ours)	5.33	52.6	5.43	53.6	**0.51**	**50.2**
	Original FPPVS	**5.13**	64.1	**5.12**	64.0	0.62	62.4
	2022 adaptation	5.14	**51.2**	5.32	**53.0**	0.63	65.2
	Greedy	6.12	100.5	6.34	104.1	6.52	105.0
	Random	6.03	97.6	6.14	99.4	6.41	104.0

Observations.

- **Bandwidth.** Updated FPPVS holds about half the link bandwidth per running SFC of greedy and random (48–49% less) and 19% less than the original FPPVS. Against the 2022 adaptation it is about even: 0.5% less on average, lower at $\rho = 0.9$ and $\rho = 1.0$, and slightly higher at $\rho = 1.1$. Keep in mind that updated FPPVS carries 1.5–1.9 times as many requests as the 2022 adaptation at the same time, including harder ones that the other method rejects.
- **Instances.** Updated FPPVS starts 0.51–0.57 new VNF instances per accepted request, about 14% fewer than the original FPPVS and the 2022 adaptation and about 92% fewer than greedy and random (about 6.6, one per VNF, since they do not share).
- **Compute.** Against greedy and random it needs about 13% less compute per service, because it shares instances. Against the original FPPVS and the 2022 adaptation it uses about 4% more compute per service. Those two methods accept far fewer requests, most likely the easier ones, so their per-request averages come from an easier set. The difference is small next to the 36–38 points of extra acceptance.
- **Latency.** Mean latency of accepted requests is 46–53 ms for updated FPPVS, 62–70 ms for the original FPPVS and the 2022 adaptation, and 104–109 ms for greedy and random. All are below the 100–150 ms limits. The 2022 adaptation places the chain strictly in sequence (no parallel branches), so every monitor adds to the critical path.
- **Decision time.** Updated FPPVS needs 9–13 ms per request, including the CNN forward pass and occasional retries. Greedy and random take 2–3 ms. Training one updated FPPVS model with 2000 episodes takes about 60 s on the CPU.

D. Effect of the algorithm alone

Our main experiments use a different arrival process from our first version (modification M1). To separate the effect of the algorithm from the effect of the workload, we also ran the updated placement logic on the original workload: one arrival per time unit, 100 requests, with almost nothing released during the run (Table 9). The updated placement reaches 70.4%, up from 10.4% for the original implementation. Acceptance is lower than in the Poisson runs mainly because the original stream offers far more traffic than the network can hold.

Table 9: Original workload (one arrival per time unit), 5 seeds. The upper rows are from our first implementation; the lower rows use the updated environment and placement logic.

Method (original workload, 100 requests)	Acceptance (%)
Original implementation, trained FDRL (5 seeds)	10.4 ± 5.1
Original implementation, greedy	15.8 ± 3.1
Original implementation, random	11.6 ± 3.0
Updated FPPVS placement	70.4 ± 2.9
Greedy	45.4 ± 2.1
Random	46.2 ± 2.2

E. Comparison with the original FPPVS and the 2022 adaptation

Same scenarios, same requests. Table 10 puts the three learning-based methods side by side. All three get 2000 training episodes and are tested on the same hold-out requests.

Table 10: Original FPPVS, the 2022 adaptation and updated FPPVS on identical scenarios, mean over 5 seeds. *Gain = updated FPPVS minus the better of the other two, in percentage points.

	Acceptance (%)				Bandwidth per running SFC		
Scenario	Original	2022	Updated	Gain*	Original	2022	Updated
Backbone 0.9	45.0	47.8	**91.0**	+43.2	71.2	58.4	**53.8**
Backbone 1	43.6	44.5	**86.4**	+41.9	70.2	56.5	**56.2**
Backbone 1.1	40.9	43.0	**80.8**	+37.8	66.9	57.0	**59.6**
Metro 0.9	50.2	53.1	**86.1**	+33.0	65.5	51.8	**51.1**
Metro 1	49.7	49.4	**81.3**	+31.5	64.3	52.0	**51.6**
Metro 1.1	47.2	50.1	**76.6**	+26.5	64.1	51.2	**52.6**
Other load levels							
Backbone 0.5	58.1	68.1	**99.6**	+31.6	70.2	57.6	**40.5**
Backbone 1.25	38.7	41.1	**75.2**	+34.1	68.2	57.1	**59.9**
Backbone 1.5	35.0	38.4	**65.2**	+26.8	67.7	57.8	**59.5**
Metro 0.5	64.6	74.0	**99.0**	+25.1	65.8	52.9	**41.0**
Metro 1.25	44.0	40.5	**69.2**	+25.2	63.9	52.2	**55.4**
Metro 1.5	36.7	42.2	**60.6**	+18.5	62.4	51.7	**55.9**

- Updated FPPVS accepts 27–43 percentage points more than the better of the two other learning methods in the main scenarios, and 18–34 points more at the other load levels.
- Both other methods fail in the same way. For the original FPPVS, 81–83% of rejections are link bandwidth and 17–19% are latency violations; for the 2022 adaptation the split is 63–70% bandwidth and 30–37% latency. Every one of these is found only after the whole chain has been placed, because both methods prune DCs by compute alone (limitation L3). The 2022 adaptation has more latency failures because it does not parallelize monitors.
- The original FPPVS's actor cannot tell DCs apart after global pooling (L5), so it tends to spread a chain over several DCs and uses the most bandwidth of the three. The 2022

adaptation's fully connected network keeps each DC in a fixed input position and uses less bandwidth than the original, but it still has no way to see bandwidth or latency before the chain is done.

Published numbers. Table 11 shows the results published in the FPPVS paper next to our three runs at full load ($\rho = 1$).

Table 11: Published FPPVS results (Tables 6 and 7 of the paper) next to our runs of the original FPPVS, the 2022 adaptation and updated FPPVS at load 1.0. Values come from different data and settings; see the text before comparing them.

		Published	Our runs, load 1.0		
Topology	Metric	FPPVS (paper)	Original	2022	Updated
Backbone	Acceptance (%)	88.1	43.6	44.5	86.4
	Latency (ms)	134.4	64.2	67.2	50.0
	Resource utilization (%)	75.35	40.2	42.3	82.8
Metro	Acceptance (%)	75.9	49.7	49.4	81.3
	Latency (ms)	160.1	62.2	67.2	48.6
	Resource utilization (%)	72.36	49.3	49.4	82.5

Our run of the original specification reaches 43.6% (Backbone) and 49.7% (Metro), far below the 88.1% and 75.9% the paper reports. We could not reproduce the published numbers, and we see four likely reasons:

1. **Workload.** The paper's arrival rate is not given (L1). Its numbers may correspond to a lighter load than our $\rho = 1$.
2. **Data.** The paper uses the real USAnet and Italian operator topologies and traces (L2); we use random graphs of the same size.
3. **Training budget.** The paper trains for 2000 iterations of 10 episodes with 20 SFCs each, about 400,000 SFC placements. Our models see 2000 episodes (one SFC each) on a laptop CPU.
4. **Unstated details.** Several parts of the original method are not fully described (sharing capacity, exact state encoding, final-reward scale), so our reading of them may differ from the authors' code.

Latency and utilization are also defined differently. Our latency counts link delay and VNF processing on the critical path, and the paper's values (above 100 ms) suggest it includes more delay components. Our utilization is the time-averaged share of total compute in use. These rows indicate trends only.

What the comparison does show is consistent: under the same heavy-load conditions, updated FPPVS raises mean acceptance in the main scenarios from 46.1% (original FPPVS) and 48.0% (2022 adaptation) to 83.7%. At full load, updated FPPVS (86.4% and 81.3%) is in the same range as the paper's published values (88.1% and 75.9%).

Summary of Contributions

Table 12: What comes from the original paper and what our team added.

Original paper [1]	Our team
Problem statement, system model and weighted objective	Explicit offered-load workload with Poisson arrivals (M1); resource-time objective and metrics (M4)
Algorithm 1 for s-SFC to p-SFC conversion	Python implementation and unit tests; critical-path latency evaluator
Idea of VNF sharing and SQN-LSTM forecasting	Instance-overhead compute model with reference counting and TTL release, without overbooking (M2)
Fairness-aware utility and JFI	JFI-change bonus in the reward
CNN actor-critic with FDRL and adaptive ϵ-greedy	Per-DC permutation-equivariant actor with learnable placement prior; per-agent streams and ledgers; experience-weighted FedAvg; entropy bonus
Node pruning by compute	Look-ahead mask on compute, bandwidth and latency; rollback-and-retry admission
Evaluation against RVNF, DRL-SFCP, PVFP, DPC on private data	Reproducible synthetic evaluation against the original FPPVS and a 2022 adaptation (Zhang et al.) on identical scenarios; greedy and random baselines, ablations, 5 seeds, a load sweep from 0.5 to 1.5

Conclusion

We implemented the FPPVS framework in Python and found that, as specified, it could not be reproduced and performed poorly in our hands: about 10% acceptance, below random placement. The main causes were an unstated arrival rate that overloaded the network, feasibility checks that came too late, a sharing model that could overbook capacity, and an actor network that could not tell data centers apart. We stated the workload as an explicit offered load, added a look-ahead feasibility mask, an instance-overhead model for VNF sharing, a per-data-center actor with a learnable placement prior, rollback-and-retry admission and corrected federated training.

With these changes FPPVS accepts 76.6–91.0% of requests on both topologies at heavy load (90–110% of capacity), above the 70% target in every main scenario and every seed. On the same scenarios, the original FPPVS as specified in the paper reaches only 40.9–50.2% and the 2022 adaptation 43.0–53.1%, and we could not reproduce the paper's published values with the original specification. Updated FPPVS also holds about half the link bandwidth per running service of greedy and random, 19% less than the original FPPVS and about the same as the 2022 adaptation, and it starts far fewer VNF instances than any other method. The ablation study shows that VNF sharing and the problem modifications drive these gains, while the learned part of the policy adds little beyond its prior.

Future work: (1) evaluate on the real USAnet and Italian operator topologies and traces once they are available; (2) re-implement the paper's baselines; (3) give the agent a multi-request horizon, for example by treating a batch of arrivals as one episode, so that learning can improve on the prior; and (4) add VNF migration so that long-lived chains can be repacked when links fill up.

References

[1] Q. Zhu, M. Li and A. Rezaeipanah, "Fairness-aware and parallelized SFC placement with VNF sharing in MEC-enabled IoT networks," *Computer Networks*, vol. 290, p. 112774, 2026.

[2] B. McMahan, E. Moore, D. Ramage, S. Hampson and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.

[3] Zhang *et al.*, "A dynamic planning model for deploying service function chain in fog-cloud computing," *Journal of King Saud University – Computer and Information Sciences*, vol. 34, pp. 7948–7960, 2022.

[4] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. ICML*, 2016.

[5] R. Jain, D. Chiu and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," DEC Technical Report TR-301, 1984.